

Compilers

Project 6: Advanced Features

*2nd Bachelor Computer Science
2025 – 2026*

Kasper Engelen
kasper.engelen@uantwerpen.be

For the project of the Compilers course you will develop, in groups of 3 students, a compiler capable of translating a program written in (a subset of) the C language into the LLVM intermediate language, which is in turn compiled into MIPS and binary code. The project will be completed over the semester with weekly incremental assignments. It is recommended to complete **one assignment per week!**

Concretely, you will construct a grammar and lexer specification, which is then turned into Python code using the ANTLR tool. The rest of the project will consist of developing custom code written in Python 3. This custom code will take the parse tree and turn it into an abstract syntax tree (AST). This AST will then be used to generate code in the LLVM IR language, which is in turn compiled into MIPS and binary code. At various stages of the project you will also have to develop utilities to provide insight into the internals of the compiler, such as visualising the AST tree.

On Blackboard you can find two appendices: one in which an overview of the project is given, and one which contains some information on how to use ANTLR.

The goal of this assignment is to extend your compiler to support some **advanced features**: structs, unions, file reading, and file writing.

1 Advanced features

1.1 Functionality

Extend your grammar to support the following features:

- (mandatory) You should support typedefs. These allow you to create “new” types by assigning custom names to existing types. The newly created type must not have the name of an existing type!

```
typedef <existing type> <new type>
```

You typedefs will need to support built-in types, structs, and union (if implemented).

Note: this will cause ambiguities if you declare a pointer of a custom type (`someType* ptrVar;`). The parser will not be able to see the difference between declarations and multiplications. You will need to resolve this ambiguity during the construction of your symbol table. **Note:** a `typedef` is very different from a `define`! A label introduced by `typedef` may only be used where other types are allowed to be used, while a `define` statement is much more general.

- (mandatory) User-defined structs. The structs should support members with primitive fields, arrays, enum types, as well as pointer types (pointers to primitives, structs, unions, ...). You do not have to support default values for the struct members.

```

struct ListNode {
    int value;
    char node_name[10];
    struct ListNode* next_ptr;
};

```

Note: You should only support uninitialised structs. Initialiser lists or other initialisers are not required. Your arrays do not need to support structs.

- (optional) Structs that contain other structs as a value.

```

struct Packet_Header {
    int src_addr;
    int protocol;
    char flags[4];
    enum PacketType type;
    struct Packet data; // not a pointer
};

```

- (optional) Arrays that contain structs.
- (optional) Dynamic allocation of structs (stored on the heap with `malloc` and `free`). This includes referring to structs using pointers. Accessing members through pointers also needs to be supported.

Note: you are free to choose how to implement this functionality. The only requirement is that the size and location of the memory segment is determined at runtime. LLVM IR provides `malloc` and `free`.

- (optional) Unions.

```

union NotTypeSafe {
    int as_integer;
    char as_str[30];
    float as_float;
};

```

- (optional) Function pointers. All the checks and errors that apply to ordinary pointers should also be implemented for function pointers.

```

int increment(int x, float f) {
    return x + 1;
}

int main() {
    int (*incrementPtr)(int, float);
    incrementPtr = &increment;
    int z = 5;
    z = (*incrementPtr)(z, 0.689);
    // z is equal to 6
    (*incrementPtr)(z, 32);
    // z is not equal to 7

    return 0;
}

```

```
}
```

- (optional) File reading using `fgets`. You can implement file pointers (`FILE*`) together with `fopen` and `fclose`, but this is not required. You may also just pass filenames to `fgets`:

```
char* read_file() {  
    char buffer[50];  
    fgets(buffer, 50, "some_filename.txt");  
  
    return buffer;  
}
```

- (optional) File writing using `fputs`. You can implement file pointers (`FILE*`) together with `fopen` and `fclose`, but this is not required. You may also just pass filenames to `fputs`:

```
void write_file() {  
    fputs("new content for the file", "some_file.txt");  
}
```

- (optional) Dynamically allocated strings and character buffers. These can work in combination with `printf`, `fgets`, `fputs`, etc.

Note: you are free to choose how to implement this functionality. The only requirement is that the size and location of the memory segment is determined at runtime. LLVM IR provides `malloc` and `free`. MIPS provides system calls.

1.2 Abstract Syntax Tree

You should construct an **explicit** AST from the Concrete Syntax Tree (CST) generated by ANTLR. Define your own data structure in Python to construct the AST, such that the AST does not depend on the ANTLR classes.

1.3 Visualization

To show your AST structure, provide a listener or visitor for your AST that prints the tree in the Graphviz dot format. This way it can easily be visualized. For a reference on the dot format, see <http://www.graphviz.org/content/dot-language>. There are useful tools to open dot files such as `xdot` on Ubuntu, and Visual Studio Code on MacOS.

1.4 Semantic Analysis

You should implement the following checks:

- (mandatory) Type checking for function pointers: assigning function pointers, calling functions, etc. Only if function pointers are implemented.
- (mandatory) Type checking for accessing and assigning struct members.
- (mandatory) Type checking for accessing and assigning union members. Only if implemented.

1.5 Code Generation: LLVM

Extend the code generation visitor for your AST that generates LLVM code and writes the generated code to a file. LLVM is an executable intermediary language used by popular compilers such as clang. Compiling to LLVM allows you to test your compiler early in the project, as it is closely related to the AST. More information on LLVM, as well as the language reference, can be found on its website:

- <https://llvm.org>
- <https://llvm.org/docs/LangRef.html>
- <https://github.com/numba/llvmlite>